EduTrack

Applying Data Structures and Algorithms to Academic Information Processing

DSA Project Review-1

Keshitha | KL University (Koneru Lakshmaiah Education Foundation)

Java 21 | 6 algorithm modules | 649-record student dataset | console application

Problem Statement

Academic records are large, text-heavy and queried in many different ways. Ad-hoc scans and hand-written lookups are slow to reason about and hard to justify.

Search at scale

Locate keywords across 649 student records with 33 attributes each, with exact positions.

Repeated-text analysis

Find shared and repeated substrings between student profiles without comparing every pair of positions.

Inexact queries

Tolerate typos and transpositions by measuring how close two strings are.

Optimisation and conflicts

Plan campus tours, split intervals optimally, allocate flow, and model exam conflicts.

Data source: data/student-mat.csv (649 records, 33 attributes: demographics, study habits, grades G1-G3).

Abstract and Objectives

Abstract

EduTrack is a Java console application that processes student records with algorithms written from scratch. Six engines cover exact string matching, suffix structures, edit distance and dynamic programming, network flow, NP-completeness and approximation, and randomised and parallel techniques.

Every result can be reproduced from the main menu, and a built-in benchmark cross-checks the engines against each other.

Objectives

1. Map each academic problem to a suitable algorithm and justify the choice.
2. Implement the algorithms without java.util collections, using custom containers.
3. Demonstrate results through verifiable, repeatable console runs.
4. Compare approaches by complexity, not by unsupported timings.

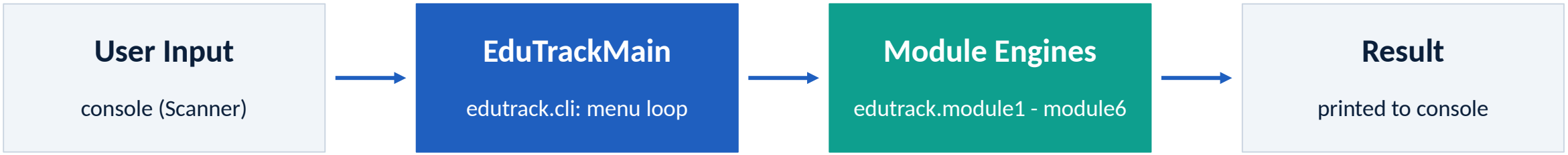
Motivation: algorithms with clear bounds make data-processing behaviour explainable.

Proposed Solution: Problems Mapped to Algorithms

Academic problem	Algorithms	Engine class	Menu route
Keyword search in records	Naive, KMP, Z, Rabin-Karp, Aho-Corasick	Module2StringEngine	3 > 1-4
Repeated-text and profile similarity	Suffix Array, Kasai LCP, longest common substring	Module2StringEngine	3 > 5, 6
Query correction	Levenshtein, Damerau-Levenshtein, sequence alignment	Module3DPEngine	4 > 1-3
Optimisation demonstrations	Matrix-Chain, Optimal BST, TSP bitmask DP, Tree DP, SOS DP	Module3DPEngine	4 > 4-9
Allocation and conflict modelling	Max-flow, matching, min-cost flow; Vertex Cover approximation	Module4FlowEngine, Module5NPEngine	5; 6

Menu route reads: main-menu option > sub-menu option. Module 1 (query routing, Miller-Rabin) and Module 6 (randomised, parallel) are also implemented and are covered in the status slide.

System Architecture and Execution Flow



Static engine methods are called directly from the CLI; there is no separate service layer.

M1
Foundational
Miller-Rabin, query routing

M2
Strings
KMP, Z, RK, Aho-Corasick, SA, LCP

M3
Dynamic programming
Edit distance, interval, bitmask, tree, SOS

M4
Network flow
Edmonds-Karp, Dinic, matching, MCMF

M5
NP and approximation
Reductions, 2-approx, FPT

M6
Randomised, parallel
QuickSort, hashing, scans

edutrack.core (support layer)

DataLoader, Student, SortSearch, TextMatch, and custom containers CustomIntArray, CustomIntQueue, CustomStringArray, CustomMatchList

Data

data/student-mat.csv and student-por.csv; the menu can switch between them (option 9).

String Algorithms: Module 2

Algorithm	Time	Role in EduTrack
Naive	O(nm)	Baseline for comparison
Knuth-Morris-Pratt	O(n + m)	Search with line and column coordinates
Z-function	O(n + m)	Linear-time cross-check
Rabin-Karp	Expected O(n + m); worst case O(nm)	Rolling hash; mod 101 versus double hash
Aho-Corasick	O(n + m + z), z = matches	Several keywords in one pass

n = text length, m = pattern length

Theory-only here: no benchmark figures are claimed. Menu 3 > 2 prints comparison counts for a chosen pattern.

Verified run (Menu 3 > 1)

Pattern "services" over the loaded corpus

317

matches, first at line 5, column 41 (offset 694)

Menu 3 > 3 compares a small modulus (101) with a double hash (10^9+7, 10^9+9). Every hash hit is verified character by character, and spurious hits are counted.

Suffix Structures: Suffix Array and LCP

Real output for "banana" (Menu 3 > 5)

i	SA[i]	LCP[i]	Suffix
0	5	0	a
1	3	1	ana
2	1	3	anana
3	0	0	banana
4	4	0	na
5	2	2	nana

The largest LCP value is 3 (row 2), so the longest repeated substring of "banana" is "ana".

Construction

Prefix doubling with merge-sort ranking: O(n log^2 n). This is not SA-IS.

Kasai LCP

Linear time, O(n), given the suffix array.

Used for

Longest repeated substring, and longest common substring of two student profiles (joined with a sentinel character).

Theory only

Suffix tree, suffix array and suffix automaton are compared in text (Menu 3 > 7); only the suffix array is implemented.

Advanced Dynamic Programming: Module 3

Edit distance

O(nm) full table

"kitten" to "sitting" = 3; Damerau "ca" to "ac" = 1

Sequence alignment

Needleman-Wunsch, Smith-Waterman: O(nm)

Global alignment of ACGTTCGAA and AGCTGCA scores 4

Interval DP

Matrix-Chain and Optimal BST: O(n^3)

Chain 10x30, 30x5, 5x60: 4500 multiplications, ((A1 x A2) x A3)

Bitmask DP: TSP

O(2^n * n^2)

6 campus nodes: minimum tour cost 101, path 0-5-4-3-2-1-0

Tree DP

O(n) passes

7-node tree: diameter 5, centroid [0], max independent set 4

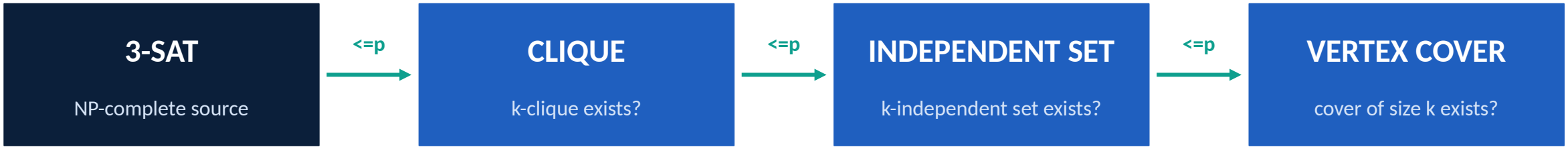
SOS DP

O(k * 2^k) for k bits

4-bit bundle: SOS[1111] = 136, SOS[0101] = 14

All results above are real outputs from the menu (Module 3 sub-menu uses built-in sample inputs). TSP here is solved by the bitmask DP, so it is an actual TSP solver on a 6-node distance matrix.

Module 5: NP-Completeness and Reductions



Implemented

`reduce3SatToClique`

Implemented

`reduceCliqueToIndependentSet`

Explained only

complement of an independent set is a cover

SUBSET-SUM and KNAPSACK

Both are weakly NP-hard number problems. SUBSET-SUM reduces to the KNAPSACK decision problem (weights = values = numbers, target = capacity).

Code: verifySubsetSum (certificate check) and approximateKnapsack (greedy). The reduction itself is theory.

HAM-CYCLE to TSP

Give edges weight 1 and non-edges weight 2: a Hamiltonian cycle exists iff a tour of cost n exists.

Code: TSP bitmask DP and Hamiltonian-path detection (Module 3). The reduction itself is theory.

A polynomial-time reduction A <=p B shows B is at least as hard as A, so reductions let us compare the difficulty of problems.

Module 5: Classes, Optimisation and Approximation

P, NP, NP-complete

P: decidable in polynomial time.

NP: a proposed solution (certificate) is checkable in polynomial time.

NP-complete: in NP and every NP problem reduces to it.

Code: verifyVertexCover, verifySubsetSum, verify3SAT

Decision vs optimisation

Decision: is there a cover of size at most k? (yes or no)

Optimisation: find the smallest cover.

NP-completeness is defined on decision problems; optimisation versions are NP-hard.

Theory

Approximation ratio

An algorithm with ratio r returns a solution within a factor r of the optimum on every input (cost at most r x OPT for minimisation).

Code: Vertex Cover 2-approx, greedy Knapsack, FPT vertex cover

Vertex Cover 2-approximation via maximal matching

Pick any uncovered edge (u, v) → Add both u and v to the cover C → Delete all edges touching u or v → Repeat until no edges remain → **|C| = 2|M| <= 2 x OPT**

Why: matched edges share no vertex, so any cover needs at least |M| vertices; C has exactly 2|M|. Verified run (Menu 6 > 3): |M| = 3, |C| = 6 on the 6-student conflict graph. The code does not compute OPT.

Implementation Demonstration (1 of 2)

EduTrack | actual console run | OpenJDK 21

```
$ java -cp bin edutrack.cli.EduTrackMain
================================================================================
          EDUTRACK: ADVANCED EDUCATIONAL ANALYTICS & ALGORITHM ENGINE
  Active Corpus: student-mat.csv (/e/data/student-mat.csv) | Total Students Loaded: 649
================================================================================
```

Step 1: Launch EduTrack

java -cp bin edutrack.cli.EduTrackMain loads the dataset (649 students) and opens the menu loop.

EduTrack | actual console run | OpenJDK 21

```
  1. Dataset Analytics & Record Inspection [student-mat.csv]
  2. Module 1: Foundational Engine (Query Routing, Design Matrix, Miller-Rabin)
  3. Module 2: String Algorithms (KMP, Z-Algorithm, Rabin-Karp, Aho-Corasick, SA, LCP)
  4. Module 3: Advanced Dynamic Programming (Alignment, Interval, Bitmask, Tree DP, SOS)
  5. Module 4: Network Flow & Reductions (Edmonds-Karp, Dinic, Min-Cut, Matching, MCMF)
  6. Module 5: NP-Completeness & Approximation (Witnesses, Reductions, 2-Approx, FPT)
  7. Module 6: Randomised & Parallel Algorithms (Las Vegas/Monte Carlo, QuickSort, Scan)
  8. Execute Comprehensive 6-Module Automated Verification Benchmark
  9. Switch Dataset (student-mat.csv <-> student-por.csv)
  10. Exit EduTrack System
================================================================================
Enter your choice [1-10]: 3
```

Step 2: Select an Academic Operation

Ten options route to the six module engines, dataset tools and the benchmark.

EduTrack | actual console run | OpenJDK 21

```
Select Option [1-7]: 1
Enter attribute/word to locate (default 'services'): services
[+] Located 317 matches for "services" in 15867 us.
    [Line: 5, Col: 41, Absolute Offset: 694] -> "services"
    [Line: 7, Col: 32, Absolute Offset: 968] -> "services"
    [Line: 10, Col: 32, Absolute Offset: 1396] -> "services"
    [Line: 13, Col: 32, Absolute Offset: 1826] -> "services"
    [Line: 14, Col: 41, Absolute Offset: 1984] -> "services"
    [Line: 18, Col: 32, Absolute Offset: 2543] -> "services"
    ... (311 more coordinates recorded)
```

Step 3: Execute String Matching

KMP finds 317 occurrences of "services" and maps each to a line and column (Menu 3 > 1).

Reproduce steps 1-3

- Run edutrack.cli.EduTrackMain (Eclipse: Run As > Java Application)
- Enter 3 for Module 2, then 1 for KMP
- Type the pattern: services

Implementation Demonstration (2 of 2)

EduTrack | actual console run | OpenJDK 21

```
Enter string to build Suffix Array & LCP for (default 'banana'): banana
>> Suffix Array & Kasai LCP Array for "banana":
  i   SA[i]   LCP[i]   Suffix String
-------------------------------------
  0   5       0        a
  1   3       1        ana
  2   1       3        anana
  3   0       0        banana
  4   4       0        na
  5   2       2        nana
```

Step 4: Analyze Text Using Suffix Structures

Suffix array and Kasai LCP for "banana" (Menu 3 > 5).

EduTrack | actual console run | OpenJDK 21

```
Select Option [1-10]: 1
>> Levenshtein Edit Distance ("kitten" <-> "sitting"): 3
>> Damerau-Levenshtein with Adjacent Transposition ("ca" <-> "ac"): 1
   Note: Classic Levenshtein requires 2 ops (del+ins); Damerau requires 1 op.
```

Step 5: Calculate Edit Distance

Levenshtein gives 3 for kitten/sitting; Damerau counts a swap as 1 (Menu 4 > 1).

EduTrack | actual console run | OpenJDK 21

```
Select Option [1-10]: 4
>> Matrix-Chain Multiplication for Chain Dimensions [10x30, 30x5, 5x60]:
   Minimum Scalar Multiplications: 4500
   Optimal Parenthesization Order: ((A1 x A2) x A3)
Select Option [1-10]: 6
>> Solving 6-Campus Inspection Tour via TSP Bitmask DP O(2^N * N^2):
   Minimum Tour Cost: 101
   Optimal Tour Path: 0 -> 5 -> 4 -> 3 -> 2 -> 1 -> 0
```

Step 6: Execute Dynamic Programming

Matrix-Chain: 4500 multiplications; bitmask TSP: cost 101 (Menu 4 > 4, then 4 > 6; two runs shown together).

Complexity Analysis

Algorithm	Time	Algorithm	Time
Naive matching	`O(nm)`	**KMP / Z-function**	`O(n + m)`
Rabin-Karp	`avg O(n + m), worst O(nm)`	**Aho-Corasick**	`O(n + m + z)`
Suffix array (doubling + merge sort)	`O(n log^2 n)`	**Kasai LCP (given SA)**	`O(n)`
Levenshtein (full table)	`O(nm)`	**Needleman-Wunsch / Smith-Waterman**	`O(nm)`
Matrix-Chain Multiplication	`O(n^3)`	**Optimal BST (as implemented)**	`O(n^3)`
TSP bitmask DP	`O(2^n * n^2)`	**SOS DP (k bits)**	`O(k * 2^k)`
Edmonds-Karp	`O(V E^2)`	**Dinic (general bound)**	`O(V^2 E)`
Vertex Cover 2-approx (adjacency matrix)	`O(V^2)`	**Tree DP (rerooting)**	`O(n)`

Bounds are the standard analytical bounds for the implemented methods; no measured benchmark results or accuracy percentages are claimed. n = text or input size, m = pattern length, V and E = vertices and edges.

Implementation Status and Rule Compliance

Implemented and runnable

- Module 1: Miller-Rabin, query routing
- Module 2: Naive, KMP, Z, Rabin-Karp, Aho-Corasick, SA, Kasai LCP
- Module 3: edit distance, alignment, Matrix-Chain, OBST, TSP, Tree DP, SOS
- Module 4: Edmonds-Karp, Dinic, matching, MCMF, scheduling
- Module 6: QuickSort, hashing, reservoir, scans

Partially implemented

- Module 5: 3-SAT to CLIQUE and CLIQUE to INDEPENDENT SET constructions only
- Vertex Cover 2-approx, greedy Knapsack, FPT cover, certificate checkers

Theory only

- INDEPENDENT SET to VERTEX COVER (explained)
- SUBSET-SUM and KNAPSACK; HAM-CYCLE to TSP reductions
- Suffix tree and automaton; PTAS, FPTAS, APX taxonomy (text output)

Course rule check: no java.util.* inside the algorithm engines

- Source search of module1 - module6 and core: no java.util imports or qualified use; engines use custom containers plus java.lang (Math, StringBuilder).
- java.util.Scanner appears once, in the CLI class EduTrackMain (outside the engines); DataLoader and the CLI use java.io for file input.
- The "zero java.util" line printed by menu option 8 is a fixed message, not a runtime check; the claim rests on the source search above.

Limitations, Future Scope and Conclusion

Current limitations

- Module 3 and 5 demos use built-in sample inputs; typed input is limited (e.g. KMP pattern, suffix-array string)
- Suffix array is O(n log^2 n), not linear-time SA-IS
- Console interface only; no automated unit-test suite
- Reductions beyond 3-SAT to CLIQUE to INDEPENDENT SET are not coded

Future scope

- Accept user-supplied inputs for the DP and NP demos
- Add a linear-time suffix array (SA-IS)
- Implement the remaining reductions as executable code
- Add unit tests and timed benchmarks on both datasets

Conclusion

EduTrack applies appropriate Data Structures and Algorithms to academic information-processing problems, and its implementation can be demonstrated through clear, verifiable execution results.